忙碌海狸 Busy Beaver

在所有 n 狀態的圖靈機中，只看最後會停下的那些，停下前跑最多步 (或寫最多個 1) 的成績就是 BB(n)。

01 從「冠軍存在」到「連證明都不可能」

n 張卡片的機器只有有限多台

冠軍存在 N2

剔除不停的機器，有限個有限成績必有最大值。

算不出來 N3

若能算出 BB(n)，跑滿 BB(n) 步就能判定停機，矛盾。

長得最快 N4

任何可計算函數，n 夠大後都被 BB 超越。

證不出來 N5

某些 BB(k) 的值 ZFC 無法證明，已知 $k \leq 254$。

每個 BB(n) 都有確定且唯一的值

沒有程式能全部算出，有些連證明都不可能

02 冠軍步數的爆炸

BB(4) 步數 107 → BB(5) 步數 約 4718 萬

BB(5)：1989 年找到冠軍機器，2024 年才證畢，相隔 35 年。

BB(6)：卡在考拉茲類怪獸機器，下界已大於 $10\uparrow\uparrow10\uparrow\uparrow10\uparrow\uparrow8$。

03 BB 勝過可計算函數的兩種強度

	機制一	機制二
結論	BB 無限多次贏	某個 N 起每項都贏
f 能否贏回	可以，時贏時輸	不行
證明手法	沿用停機判定流程	把 f 裝進參賽機器

04 必記術語

停機問題：判斷程式會不會停，無通用解法

可計算函數：同一程式能算出所有 n 的值

頑抗機器：久跑未停、須另證不停的機器

怪獸機器：規則已看懂，卻是未解數學難題

獨立於 ZFC：ZFC 證不出真，也證不出假

05 你曾混淆的地方

每個值都確定存在，不等於可計算：可計算要求同一個程式算出所有 n。

想先找公式 f 當上限再求 BB：這樣的 f 不存在，否則 BB 就可計算。

延伸主題：哥德爾不完備定理、考拉茲猜想

忙碌海狸尺：寫成機器所需的最少狀態數，艾狄胥 15、哥德巴赫 27、黎曼 744。

名稱由來：拉多 1962 年提出，通行解讀取自英語 busy beaver「拚命工作的人」。

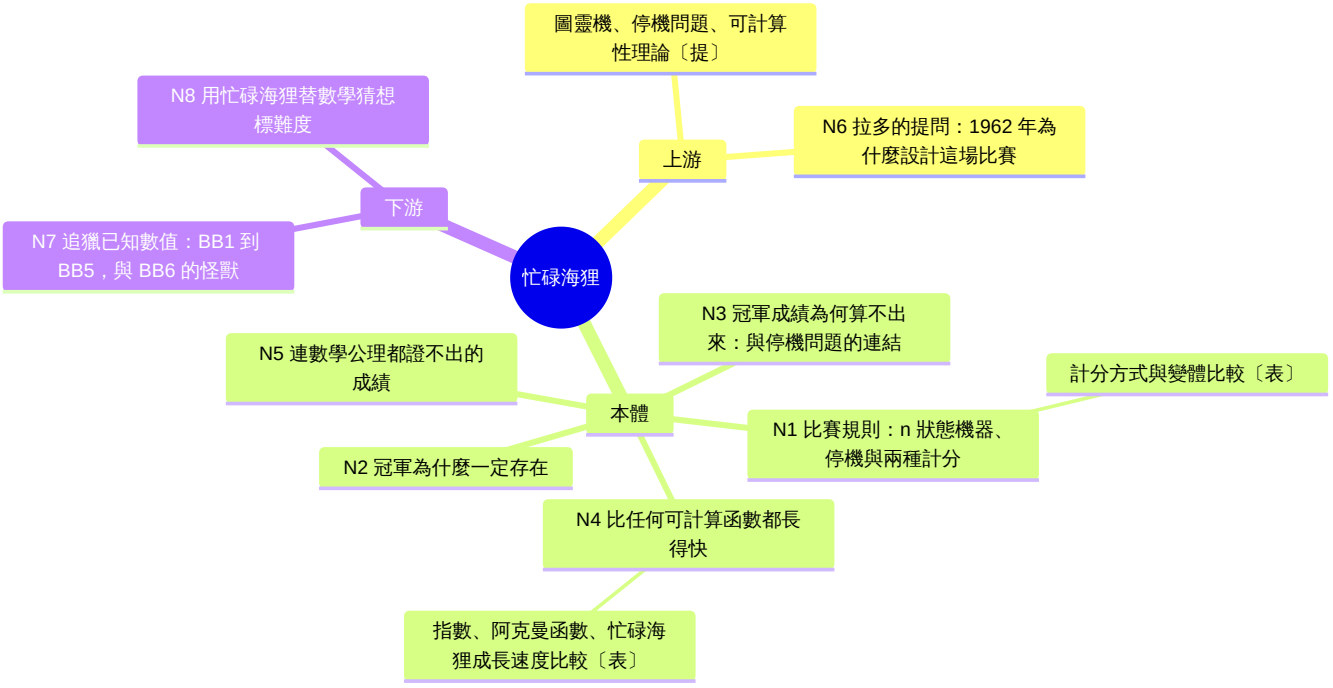
討論主題	學科	日期
忙碌海狸 (Busy Beaver)	理工／資訊 (可計算性理論)	2026-10-15

一句話定義

忙碌海狸是一場極簡計算機的比賽：在所有只有 n 個狀態的圖靈機中，只看最後會自己停下的那些，誰停下前忙得最久 (跑最多步或寫下最多個 1)，牠的成績就是 BB(n)。

脈絡地圖

判定：深題，本體有「機器有限所以冠軍存在」「冠軍成績無法計算」「成長快過任何可計算函數」「某些成績無法被證明」四段



編號	節點	欄位	備註
N1	比賽規則：n 狀態機器、停機與兩種計分	本體機制	
N2	冠軍為什麼一定存在	本體機制	
N3	冠軍成績為何算不出來：與停機問題的連結	本體機制	需補：停機問題 (已補)；需補：「可計算」與「有確定值」的差別 (已補)
N4	比任何可計算函數都長得快	本體機制	「可計算」定義已於 N3 補上

編號	節點	欄位	備註
N5	連數學公理都證不出的成績	本體機制	
N6	拉多的提問：1962 年為什麼設計這場比賽	起源與時點背景	
N7	追獵已知數值：BB(1) 到 BB(5)，與 BB(6) 的怪獸	發展與應用	
N8	用忙碌海狸替數學猜想標難度	發展與應用	

節點講解

N1 比賽規則：n 狀態機器、停機與兩種計分 (本體機制)

直覺：想像一條向左右無限延伸的紙帶，切成一格一格，一開始每格都寫著 0。一隻海狸 (讀寫頭) 站在其中一格上，手裡有一疊卡片，每張卡片就是一個狀態，上面只寫兩件事：「腳下是 0 時怎麼做」與「腳下是 1 時怎麼做」。比喻的主要失效處：真的海狸有記憶與判斷力，這隻海狸唯一的「記憶」就是手上正拿著哪張卡片，以及紙帶上已經寫下的符號。

機制：每一步只做三件事：在腳下這格寫 0 或 1、往左或往右走一格、換拿某一張卡片。其中一個去處是特殊的「停機」(H)，拿到它機器就結束。比賽固定從全 0 紙帶、拿著卡片 A 開始。n 個狀態就是 n 張卡片，每張兩條指令，所以一台 n 狀態機器就是一張 2n 格的規則表。

為什麼一定要「會停下」才算數：如果不要求停下，一台「看到 0 就寫 1 往右走，換回同一張卡」的單狀態機器就能永遠寫 1、永遠跑下去，比賽失去意義。所以只有最後會自己停下的機器能參賽，永遠跑下去的機器全數淘汰。忙碌海狸的趣味在於：要在「會停」的限制下拖得越久越好。

例子：2 狀態冠軍

狀態	腳下是 0	腳下是 1
A	寫 1，右移，換 B	寫 1，左移，換 B
B	寫 1，左移，換 A	寫 1，右移，停機

這台機器跑 6 步後停下，紙帶上留下 4 個 1 (逐步軌跡見圖 1)。所有 2 狀態機器中，沒有任何一台能在停下前跑超過 6 步或寫超過 4 個 1。



第 6 步遇到「B 看到 1」才停機，紙帶留下 4 個 1

圖 1：2 狀態冠軍的逐步軌跡

看圖重點：讀寫頭在左右來回中把 0 逐格填成 1；第 6 步碰到「B 看到 1」才停機，留下 4 個 1。

細節：兩種計分方式

名稱	記號	計什麼	n = 2 的冠軍成績	備註
1 的個數	Σ(n)	停下時紙帶上 1 的數量	4	拉多 1962 年原始定義
步數	S(n)	停下前總共走了幾步	6	現在 bbchallenge 等社群說 BB(n) 時多指這個
多符號版	BB(n, k)	紙帶可用 k 種符號，而非只有 0 與 1	(k = 2 即上兩列)	讓「卡片數」與「符號數」分開比較

兩種計分的冠軍不一定是同一台機器，但兩者成長的方式極為相似。本單元說 BB(n) 時，若沒特別註明，指的是步數 S(n)。

N2 冠軍為什麼一定存在 (本體機制)

直覺：一場報名人數有限的跑步比賽，只要把中途棄賽、永遠跑不到終點的人剔除，剩下每個人都有一個有限的完賽時間，名單有限，時間有限，就一定排得出第一名。忙碌海狸的冠軍存在，靠的是同一個道理。比喻的主要失效處：跑步比賽裁判看得出誰棄賽，忙碌海狸裡「誰永遠跑不完」卻不一定看得出來 (這是 N3 的主題)。

機制：論證分三步。

1. **參賽機器只有有限多台**：規則表有 2n 格，每格要選「寫 0 或 1」(2 種)、「左或右」(2 種)、「換到 n 張卡片之一或停機」(n + 1 種)，所以每格有 4(n + 1) 種寫法，整張表共有 (4n + 4) 的 2n 次方種。停機 H 只是一個「去處」，它本身沒有規則可寫，所以算進每格的選項 (底數的 n + 1)，卻不算進表格的格數 (指數仍是 2n)。這個數字是固定 n 時「所有可能的規則表」的精確總數，同一個 n 下怎麼排都不會超出；其中還包含只是把卡片改名、或左右鏡像的重複機器，實質不同的機器更少。
2. **剔除永遠不停的機器**：剩下的每台機器都在某個有限步數停下，所以每台都有一個有限的成績。
3. **有限多個有限數一定有最大值**：把這些成績排一排，最大的那個就是 BB(n)。

所以對每一個 n，BB(n) 都是一個確定的、有限的正整數。

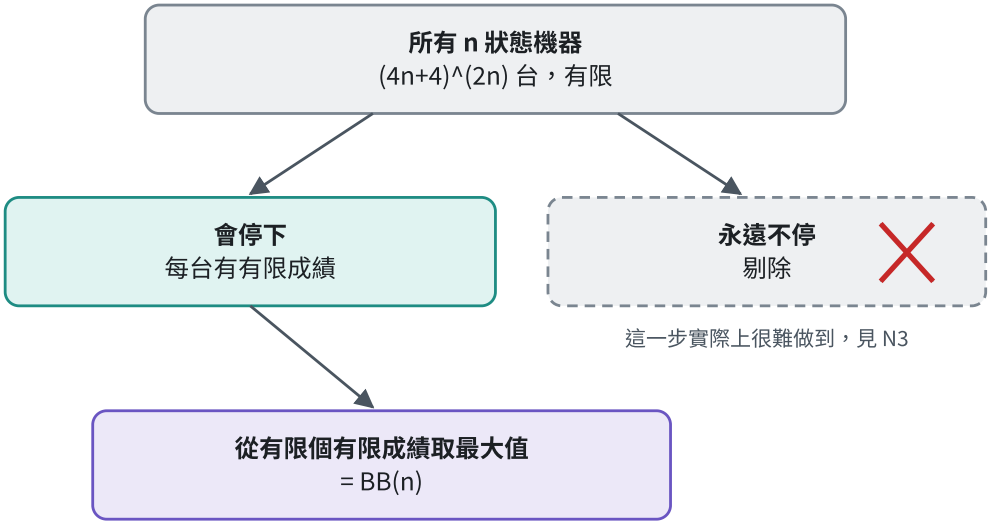


圖 2：冠軍存在的三步篩選

看圖重點：冠軍存在只需要「有限」；真正的難處在中層分支，不在取最大值。

邊界：最容易卡住的地方：「有些機器會跑無窮多步，最大值不就變成無限大了嗎？」不會，因為那些機器在第 2 步就被剔除，根本不參與排名。BB(n) 是「會停的機器裡最大的步數」，永遠不停的機器步數不算。

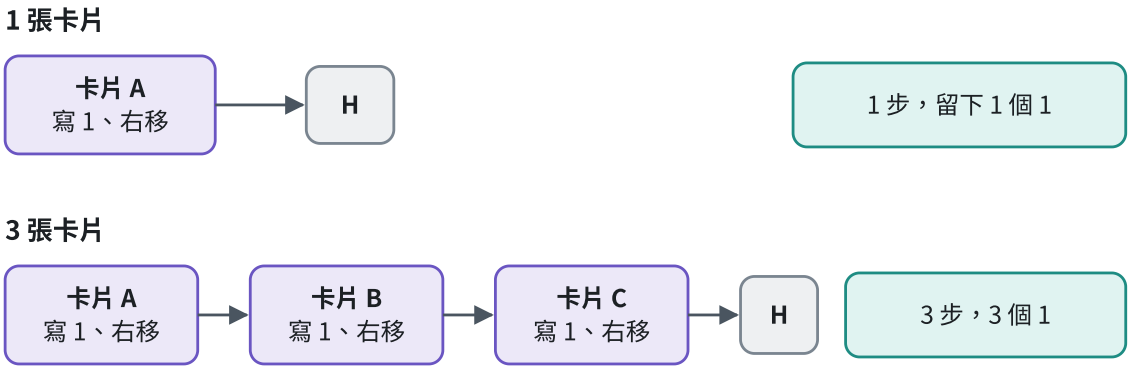
細節：參賽人數

n	機器總數 (4n + 4)^(2n)
1	64
2	20,736
3	16,777,216
4	25,600,000,000
5	約 6.3 × 10^13
6	約 2.3 × 10^17

機器數量增加得很快，但對每個 n 都仍是有限的，這就足以保證冠軍存在。這份計算也說明了 BB(n) 隨 n 不會減少：多一張卡片時，原本的機器都還能照舊參賽 (新卡片不用到即可)，所以冠軍成績只會持平或變大。

多一張卡片增加的是什麼：動作的種類始終只有寫 0／1、左／右兩類，n 變大時並沒有新增動作。增加的是兩樣東西：表格多了兩格 (指數 2n 變大)，以及每格「下一步換到哪張卡」的選項多一個 (底數 4(n + 1) 變大)。對機器的行為來說，關鍵在後者代表的意義：卡片是機器唯一的內部記憶，卡片越多，機器能區分的「情境」越多，卡片之間能串成的路線就越複雜。1 張卡片的機器只能重複同一個反應；2 張卡片可以交替；卡片再多一些，就能組出迴圈 (依同一組卡片反覆執行)，甚至圈中有圈，例如內圈每繞一次就把紙帶上的計數加一，外圈再依計數決定內圈要繞幾次。這種層層套疊的能力，才是冠軍成績隨 n 暴增的來源，動作種類始終沒變。

卡片是「編號的步驟」，不是「動作種類」 (圖 4)：動作的組合確實只有四種 (寫 1 右移、寫 0 右移、寫 1 左移、寫 0 左移)，但卡片並不是按動作分類的。每張卡片都是一個有編號的步驟，裡面寫兩條指令 (看到 0 時、看到 1 時)，每條指令除了動作，還指定「下一步換到哪張卡」。所以兩張卡片的動作可以一模一樣，差別只在箭頭指向哪裡。最簡單的例子：只有 1 張卡片時，「看到 0 就寫 1、右移」之後只能選擇停機 (只寫 1 個 1) 或回到自己 (永遠寫下去、被淘汰)；有 3 張卡片時，讓 A 指向 B、B 指向 C、C 指向停機，三張卡片動作相同，機器卻能「數到 3」：寫 3 個 1 再停。卡片的作用是讓機器記得自己進行到第幾步。再進一步，箭頭也可以指回前面的卡片 (例如 N1 的 2 狀態冠軍中，B 看到 0 時換回 A)，這就形成迴圈，前面的卡片被重複使用，配合紙帶上已寫的內容決定何時跳出，步數因此能遠多於卡片數。



三張卡片的動作完全一樣，不同的只有箭頭：下一步換到哪一張

圖 4：1 張卡片與 3 張卡片的串接

看圖重點：卡片的差別在箭頭 (下一步去哪)，動作可以相同。

更貼切的比喻：冒險遊戲書：每張卡片像冒險遊戲書的一頁，頁面上寫著「若腳下是 0，做某動作，翻到第 B 頁；若腳下是 1，做另一動作，翻到第 H 頁」。這個比喻點出三件事：第一，每頁有兩個分支，下一頁去哪由紙帶上的符號決定，沒有預先排好的固定順序；第二，同一頁可以反覆翻回來，卡片不會被用掉；第三，整個過程沒有任何隨機或選擇，給定規則表與紙帶，下一步完全確定。比喻的主要失效處：遊戲書是讀者自己選分支，這裡是紙帶上的 0 或 1 替機器「選」。以 N1 的 2 狀態冠軍為例 (圖 1)：卡片 B 在

第 1、3、5 步都被用到，第 1 和第 3 步腳下是 0，所以寫 1、左移、翻回 A；第 5 步腳下已經是 1，才走「寫 1、右移、停機」那條分支。同一張卡片，因為紙帶內容不同而走向不同結局，這就是機器能用少量卡片跑出許多步的原因。

用里程表理解套疊 (圖 3)：汽車里程表的每個輪子都只會一個動作：轉一格。1 個輪子只能數 10 種情況就回到原點；加第 2 個輪子，規則變成「右輪轉滿一圈，左輪才轉一格」，可數的情況變成 100；3 個輪子是 1000。新增的輪子沒有帶來新動作，它多了一層「替下一層數圈數」的記憶，效果是相乘。圖靈機的卡片可以扮演同樣的角色：幾張卡片負責內圈 (例如來回把一段 1 推長一格)，另幾張卡片負責外圈 (內圈做完一輪後，把紙帶上的計數加一，再決定要不要重來)。這個比喻的主要失效處：里程表輪子數固定，數到上限就歸零；圖靈機的計數寫在無限長的紙帶上，可以一直長大，而且外圈還能根據內圈的成果改變下一輪的長度，所以增長遠比「乘 10」劇烈。已知的冠軍步數可以看出這種爆炸：n = 1 到 5 依序為 1、6、21、107、47,176,870。

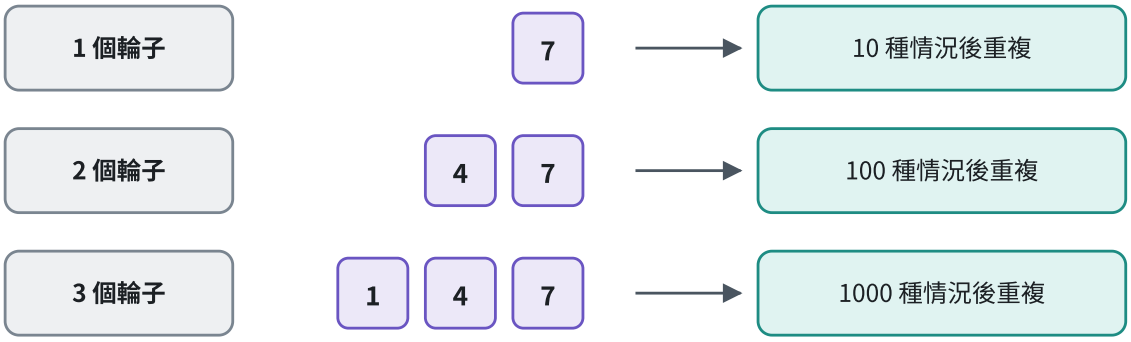


圖 3：里程表輪子數與可數範圍

看圖重點：動作不變，多一層記憶就讓可數範圍相乘。

存在與找到是兩件事：論證只保證 BB(n) 有確定的值，並沒有告訴我們怎麼把它找出來。難處卡在第 2 步：要剔除永遠不停的機器，得先判斷每台機器到底會不會停。

N3 冠軍成績為何算不出來：與停機問題的連結 (本體機制)

先補一：「有確定值」和「可計算」是兩回事：N2 證明了每個 BB(n) 都有確定的值，但數學上說一個函數是**可計算函數**，要求更強：必須存在**同一個程式**，輸入任何 n，都能在有限時間內輸出 f(n)。關鍵在「同一個程式通吃所有 n」。如果只要求「每個個別的 n 都有辦法算」，那條件形同虛設，因為對任何一個固定的 n，永遠有一個只寫著「印出這個數字」的程式 (例如 BB(4) 就對應「印出 107」)，但這種程式得先知道答案才寫得出來。忙碌海狸正是「每個值都確定存在，卻沒有一個程式能全部算出來」的經典例子。

「有限時間」的意思：在可計算性理論裡，時間用「步數」計算，不用秒數，這樣結論就和電腦快慢無關。「在有限時間內算出 f(n)」的意思是：對每一個輸入 n，程式都會在**某個**有限的步數停下，並在紙帶上留下正確答案 f(n)。這個定義刻意很寬鬆，有三點要注意：

1. **沒有統一的上限**：輸入 n 越大，可以跑越多步，步數只要對每個 n 是有限的就好。
2. **不必事先知道要跑幾步**：只要求「最後一定會停」，不要求開始前就能說出會跑多久。
3. **再久都算**：就算要跑超過宇宙年齡的步數，只要不是永遠跑下去，都算有限時間。「在合理時間內算完」是另一門學問 (計算複雜度) 在討論的事，不屬於可計算性的要求。

所以「可計算」的完整說法是：存在同一個程式，對每個 n 都會停機，停機時紙帶上寫的就是 f(n)。和它相對的「無限時間」只有一種情況：程式永遠不停。BB 之所以不可計算，跟「算得太慢」無關：無論允許程式跑多久，都沒有任何程式能對每個 n 停下並給出正確的 BB(n)。

先補二：停機問題：**停機問題**問的是：能不能寫一個判定程式，輸入任意一個程式 (連同它的輸入)，它都能在有限時間內正確回答「這個程式最終會停，還是會永遠跑下去」。圖靈 (Alan Turing) 在 1936 年證明這樣的判定程式不存在。證明的想法是自我指涉：假設判定程式 D 存在，就能寫出一個「唱反調」的程式 X，它把自己交給 D 判斷，D 說 X 會停，X 就故意進入無窮迴圈；D 說 X 不會停，X 就立刻停下。不論 D 怎麼回答都答錯，所以 D 不可能存在。要注意這是原理上的不可能，跟電腦快慢無關：問題不在跑得不夠快，而在「還沒停」的機器，光是等下去永遠無法確定它是不是真的不會停。

直覺：N2 的篩選卡在第 2 步：要剔除永遠不停的機器，就得知道每台機器會不會停。如果有人真的能算出 BB(n)，等於手上握有一個「最多等這麼久」的時限，第 2 步就變得可以做到。

機制 (圖 5)：假設存在一個能算出 BB(n) 的程式，那麼要判斷任意一台 n 張卡片的機器 M 會不會停 (從全 0 紙帶開始)，只要：

1. 用那個程式算出 BB(n)。
2. 讓 M 實際跑 BB(n) 步。
3. 若 M 已經停了，答案是「會停」；若還沒停，答案是「永遠不停」。理由：如果 M 會在更晚的某一步停下，它的步數就超過 BB(n)，打敗了冠軍，這與 BB(n) 是最大值矛盾。

這樣就得到一個能判定停機的方法，但停機問題已被證明無法判定 (從全 0 紙帶出發的版本同樣無法判定)，所以一開始的假設不成立：**不存在能算出所有 BB(n) 的程式，BB 是不可計算函數。**

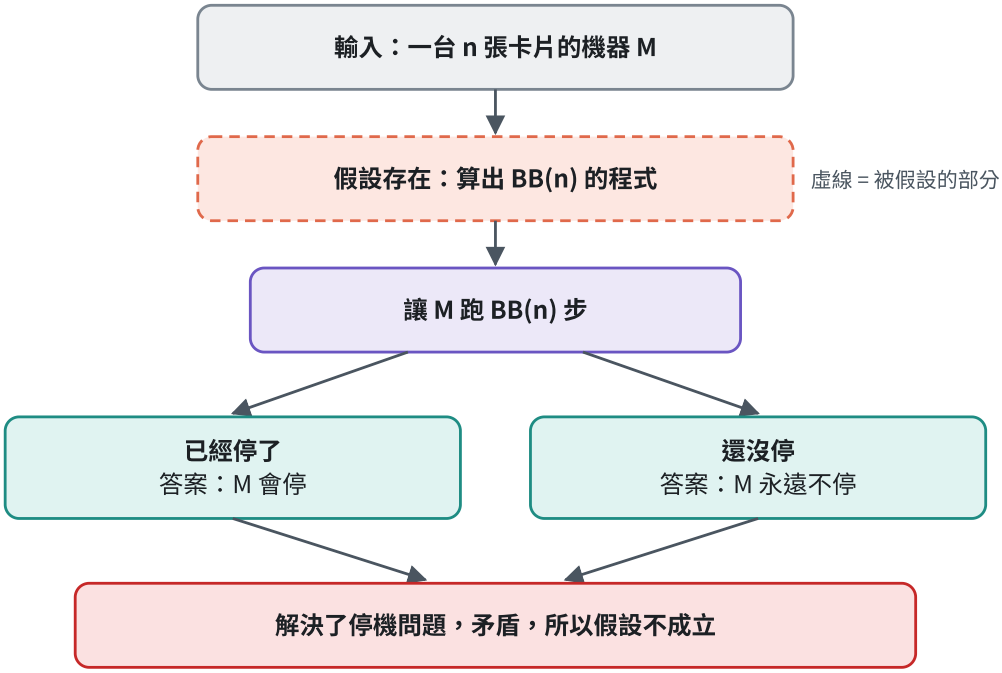


圖 5：若能算出 BB(n)，就能判斷停機

看圖重點：虛線方框是唯一的假設；整個流程一旦成立就能判定停機，所以這個假設必然是錯的。

有嚴格定義，卻沒有計算公式：BB(n) 在數學上定義得非常精確：

$$BB(n) = \max\{s(M) : M \text{ 是 n 狀態、從全 0 紙帶出發會停機的圖靈機}\}$$

其中 s(M) 是 M 停機前走的步數。這行定義沒有任何模糊之處，每個 n 都對應唯一的值 (N2)。但它屬於「描述答案是什麼」的定義，不是「照著算就能得到答案」的公式。BB(n) 不可能有像 2 的 n 次方那樣的**封閉公式**，理由直接來自上面的機制：任何由加減乘除、次方、遞迴等可以一步步執行的運算組成的公式，都能寫成一個程式，那 BB 就成了可計算函數，與停機問題矛盾。

窮舉為什麼也不夠：實際確定某個 BB(n) 的做法確實從窮舉開始：列出全部 (4n + 4)^(2n) 台機器 (先刪去改名、鏡像等重複者)，逐台執行。會停的機器很好處理，跑到停為止記下步數即可。真正的難處在剩下那些跑了很久還沒停的機器 (稱為**頑抗機器**，holdout)：光靠繼續跑永遠無法確認它們不會停，必須逐台**證明**它不會停。窮舉機器是有限的苦工，證明不停機卻沒有一個通用方法，而且也不可能有：如果有一套方法能對每個 n 自動證明所有不停機的機器，把它和窮舉合起來就成了計算 BB 的程式。所以確定 BB(n) 是「窮舉＋逐台證明」，n 越大，需要的證明手法越新、越難 (BB(5) 如何完成見 N7；某些 n 連證明都不可能存在，見 N5)。

邊界：不可計算指的是「沒有一個程式能對所有 n 算出 BB(n)」，不代表每個值都永遠查不到。BB(1) 到 BB(5) 都已經確定 (見 N7)，那是數學家針對個別的 n，用人力、電腦與證明逐台機器解決的，每往上一個 n 都要想新的辦法。

N4 比任何可計算函數都長得快 (本體機制)

直覺：把每個可計算函數想成一位有固定訓練菜單 (程式) 的跑者，忙碌海狸則是一場不斷擴大名額的比賽。任何跑者的菜單本身都能被寫成一台參賽機器，所以只要比賽名額夠大，那位跑者就一定被收進參賽名單，而冠軍成績至少跟他一樣好，甚至更好。比喻的主要失效處：真實比賽不會因為名額變大就自動收編所有跑者，這裡靠的是「任何程式都能改寫成圖靈機」。

f 是誰：BB 的對手，不是 BB 的算法：本節的 f 指任何一個普通的可計算函數，例如 n^2、2 的 n 次方、阿克曼函數，它是被拿來跟 BB 比賽的對手，和 BB 本身的計算過程無關。f 是**一條固定的規則** (一個固定的程式)，輸出 f(n) 會隨 n 改變，但規則本身不變。以 f(n) = 2 的 n 次方為例，規則永遠是「從 1 開始，乘 2 共 n 次」：n = 2 時得 4，n = 5 時得 32，用的是同一份指示。BB 則沒有這樣的固定規則：BB(2) = 6 是逐一檢查 20,736 台 2 狀態機器得到的，BB(3) = 21 又得另外檢查一千六百多萬台並逐台證明，每個 n 都要重新來過。換句話說，BB 本身沒有「它的 f」，這正是 N3「BB 不可計算」的意思。以 n = 2 為例，對手的成績分別是：n^2 得 4、2 的 n 次方得 4，都輸給 BB(2) = 6；阿克曼 A(2, 2) = 7 則贏過 BB(2)。這種小 n 時對手領先的情形，正是下方「最終」一詞要處理的事。

機制一：沒有任何可計算函數能永遠壓住 BB：假設有一個可計算函數 f，對所有夠大的 n 都滿足 f(n) ≥ BB(n)。那 f 就能取代 N3 流程中的 BB(n)：讓機器跑 f(n) 步，還沒停就永遠不會停。f 可以計算，於是停機問題被解決，矛盾。所以任何可計算函數都不可能當 BB 的永久上限。

「先找上限、再求 BB」為什麼行不通：一個很自然的想法是：先找一條規則 f，保證 f(n) ≥ BB(n)，再讓每台 n 狀態機器跑 f(n) 步，沒停的就淘汰，已停機者中最大的步數就是 BB(n)。這個流程在邏輯上完全正確，問題在於這樣的 f 不存在：只要它存在，上述流程就是一個能對所有 n 算出 BB(n) 的程式，與 N3 的結論矛盾 (流程結構同圖 5，把虛線方框換成 f)。機制一說的正是這件事。所以 N4 不是在描述「求 BB 的方法」，而是一條關於成長速度的定理：任何固定規則都追不上 BB，因此不可能拿來當它的上限。對單一的 n，上限當然可以存在 (例如 BB(5) 已知後，47,176,870 本身就是上限)，但那是逐台證明換來的結果，沒有一條規則能對所有 n 事先提供。

機制二：BB 最終一定超過任何可計算函數：取任一可計算函數 f，它由某台 c 張卡片的機器計算。對每個 n，組一台參賽機器：前面幾張卡片負責在紙帶上寫下 n，接著交給那 c 張卡片去算 f(n)，算出後再停機。寫出 f(n) 這麼大的答案本身就要走很多步，所以這台機器至少跑 f(n) 步、而且會停，它是合法參賽者，冠軍只會更大：BB(n + 額外卡片數) ≥ f(n)。額外卡片數是固定的常數，再對 f 稍作加工 (例如改用 f(2n)) 就能把這個常數吸收掉，得到：n 夠大以後，BB(n) > f(n)。

機制一與機制二的差別：「無限多次贏」和「從某處起永遠贏」：兩者容易看成同一件事，其實機制二更強。機制一推翻的是一個假設 (「n 夠大後 f 永遠不小於 BB」)，推翻之後得到的結論是：不論 n 走到多遠，後面總還會出現某個 n 讓 BB(n) > f(n)，也就是 BB **無限多次**超過 f，但這仍容許 f 在其他 n 上贏回來，形成時贏時輸。機制二則保證：存在某個 N，從 N 開始的**每一個** n 都是 BB 贏，f 再也贏不回來。

n	1	2	3	4	5	6	7	8	……
情形甲：BB 時贏時輸	輸	贏	輸	贏	輸	贏	輸	贏	(永遠交替)
情形乙：某處後 BB 永遠贏	輸	輸	輸	贏	贏	贏	贏	贏	(一路贏下去)

生活上的對照 (圖 6)：「不管過了多少天，以後總還會遇到下雨天」和「從某一天開始，之後天天下雨」顯然不同，前者容許中間夾著許多晴天，後者在某天之後一個晴天都沒有。機制一的結論像前者，機制二像後者。比喻的主要失效處：天氣是隨機的，BB 與 f 的大小則完全確定。

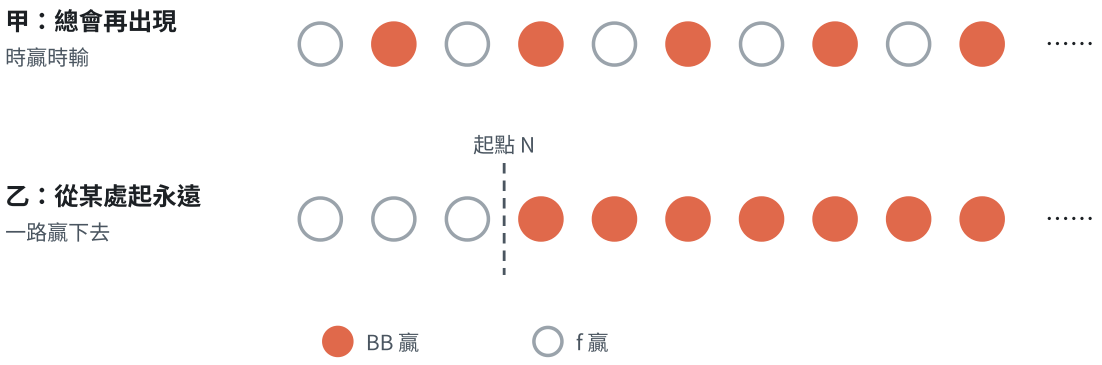


圖 6：無限多次與從某處起永遠

看圖重點：甲是「總會再遇到」，乙是「從 N 起一個空心都沒有」；機制二保證的是乙。

情形甲符合機制一的結論，卻不符合機制二；情形乙兩者都符合。機制二成立就推得出機制一的結論，反過來不行。先講機制一，是因為它直接沿用 N3 的流程、最容易接上；要得到「遲早永遠勝出」的完整說法，需要機制二的造機器論證。

本節一句話：對**每一個**可計算函數 f，都存在某個起點 N，使得 $n \geq N$ 時 $BB(n) > f(n)$。說法上有兩處要精確：一是「每一個」而非「某一個」(只比某一個 f 大毫無意義，例如 $f(n) = 0$ 誰都比它大)；二是「n 夠大之後」而非「永遠」(小 n 時 f 可能領先)。起點 N 會隨 f 而變，f 長得越快，BB 超車的位置就越晚，但一定會超車。

這個結論有什麼用：單看某一個 n，BB(n) 比 f(n) 大或小，確實取決於拿哪個 f 來比。這條定理的價值在於它與對手無關：不論拿誰來比，結局都一樣。對一般函數而言，總能找到長得更快的可計算函數壓過它 (2 的 n 次方被 n 的階乘壓過，階乘又被阿克曼函數壓過)，這條「誰比誰快」的鏈可以無限延伸；BB 則站在整條鏈的上方，鏈上每一個都被它甩開。由此得到三個實際後果：

1. **無法用公式估計 BB**：任何預測、外推或估上限的公式本身都是可計算函數，遲早會被 BB 超過，所以從 1、6、21、107、47,176,870 往後猜 BB(6) 注定失準；實際上 BB(6) 的已知下界已大於 10↑↑10↑↑10↑↑8。
2. **程式的極限無法被程式預測**：BB(n) 是「n 張卡片的程式在停機前最多能做多少事」的絕對上限，而可計算函數代表程式算得出來的一切。BB 勝過所有可計算函數，等於說沒有任何程式能事先算出程式能力的天花板。
3. **為 N5、N8 鋪路**：BB 長得這麼快，是因為少數幾張卡片就能裝進極複雜的搜尋任務，例如「找數學猜想的反例，找到才停」。正因如此，知道某個 BB(n) 的值就等於解決了某些數學難題，這是後兩節的主題。

邊界：「最終」兩個字很重要：BB 在小 n 時不一定最大。可計算的阿克曼函數 A(n, n) 在 n = 4 時已經遠遠超過 BB(5)。BB 的保證是「從某個 n 開始永遠勝出」，不是每一項都勝出。

細節：成長速度比較〔表〕

n	n^2	2 的 n 次方	阿克曼 A(n, n)	BB(n) (步數)
1	1	2	3	1
2	4	4	7	6
3	9	8	61	21
4	16	16	$2^{(2}$(2^65536)) − 3	107
5	25	32	(遠超可寫出的範圍)	47,176,870
6	36	64	(更大)	> 10↑↑10↑↑10↑↑8

讀表重點：前幾項 BB 輸給阿克曼函數，但 BB(6) 的下界已經要用迭代冪次表示；而阿克曼函數不論多快，終究是可計算函數，依機制二，BB 遲早會把它甩在後面。

N5 連數學公理都證不出的成績 (本體機制)

直覺：數學證明都從一組大家同意的起點出發，這組起點稱為**公理系統**，就像一本規則書，法官只能依書判案。現代數學最常用的規則書是 **ZFC** (策梅洛－弗蘭克爾集合論加上選擇公理)，幾乎所有數學定理都能在它裡面證明。但有些問題，這本規則書既無法證明為真、也無法證明為假，這種情況稱為該命題**獨立於** ZFC。N5 要說的是：存在某個 k，「BB(k) 等於多少」就是這樣的問題。比喻的主要失效處：法官遇到規則書沒寫的案子可以修法，數學家當然也能加公理，但加了公理後的新規則書，又會在更大的 k 遇到同樣的問題。

需要的工具：哥德爾第二不完備定理：哥德爾 (Kurt Gödel) 1931 年證明，一個夠強、又不自相矛盾的公理系統，無法在自己內部證明「我不自相矛盾」。ZFC 就屬於這種系統。

機制 (圖 7)：

1. **造一台機器 Z**：用 k 張卡片寫一個程式，從短到長逐一列出所有能從 ZFC 推出的證明，檢查每一個是否推出了矛盾 (例如 0 = 1)，找到才停機。所以 Z 會停，若且唯若 ZFC 自相矛盾。
2. **假設 ZFC 能證出 BB(k) = V**：那麼在 ZFC 裡可以實際模擬 Z 跑 V 步 (有限步的計算，ZFC 能逐步核對)，確認它沒停 (只要 ZFC 真的不矛盾，Z 就不會停)。
3. **推出 Z 永遠不停**：Z 是 k 張卡片的機器，若會停，必在 BB(k) = V 步之內停 (N3 的道理)。跑滿 V 步沒停，就永遠不會停。
4. **等於 ZFC 證明了自己不矛盾**：「Z 永遠不停」正是「ZFC 不自相矛盾」，這違反哥德爾第二不完備定理。

所以，只要 ZFC 本身不自相矛盾，它就證不出 BB(k) 的確切值。

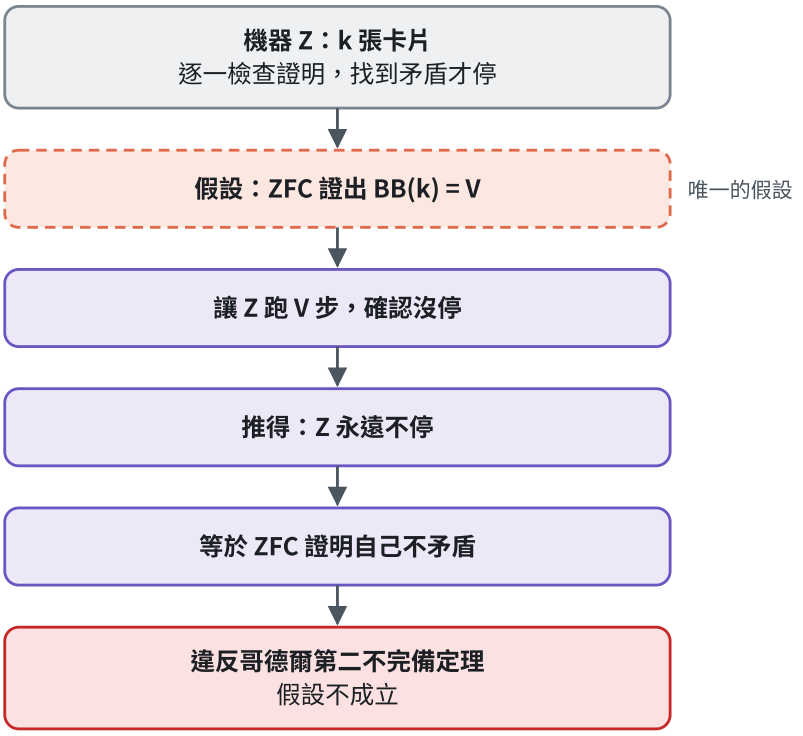


圖 7：為什麼 ZFC 證不出 BB(k)

看圖重點：虛線方框是唯一的假設；後面每一步都合法，所以錯在假設。

第 3 步為什麼成立：Z 只有 k 張卡片，所以它本身就是 k 狀態比賽的參賽者之一。BB(k) = V 的意思是「所有會停的 k 狀態機器中，最多跑 V 步」，換句話說，每一台會停的 k 狀態機器都在 V 步之內停下。若 Z 跑到第 V 步還沒停，它就不可能屬於「會停的機器」：假如它在更晚的第 V + 100 步才停，它就是一台會停、而且跑了 V + 100 步的 k 狀態機器，打破了「最多 V 步」的紀錄，與 BB(k) = V 矛盾。以 k = 2 為例：BB(2) = 6，任何 2 狀態機器只要跑到第 7 步還沒停，就一定永遠不停，例如「看到 0 就寫 1、右移、回到自己」那台。用 N2 的路跑比喻：如果確定「所有完賽者中最慢的人花了 V 分鐘」，那麼第 V 分鐘還在賽道上的人就不會是完賽者。比喻的失效處：真實路跑中，「最慢完賽時間」要等比賽結束才知道；BB(k) 則是把所有機器的整個未來都算進去的最大值，所以一旦確知 V，就能直接下結論。這也說明第 2 步的假設有多強：知道確切的 BB(k)，就握有所有 k 狀態機器的停機答案。

邊界：三種「做不到」要分清楚

說法	意思	出處
BB(k) 有確定的值	值存在且唯一，這點沒有改變	N2
BB 不可計算	沒有一個程式能對所有 n 算出 BB(n)	N3
BB(k) 獨立於 ZFC	對這個特定的 k，連「用 ZFC 證明它的值」都不可能	N5

N3 說的是「沒有通用方法」，仍容許逐個 n 靠證明解決；N5 說的是到了某個 k，連逐個證明這條路在 ZFC 裡也斷了。換用更強的公理系統可以證出 BB(k)，但新系統也有自己的 Z，會在更大的 k 卡住。

細節：k 到底多小 (bbchallenge wiki 紀錄，截至 2026-10-15)

年月	狀態數	發現者
2016-05	7910	Yedidia 與 Aaronson
2016-05	748	O’ Rear
2023-07	745	Riebel
2024-08	636	Ridenour
2025-08	432	Wade
2026-10	254	Mira (ChatGPT 協助)，Ketchersid 以 Claude 協助在 Rocq 形式化

下界是 6：BB(5) 已在 Rocq 中證明，所以第一個證不出的 k 落在 6 到 254 之間，確切位置未知。

研究從兩端夾擠：找出這個門檻是忙碌海狸研究的方向之一，但它不是唯一目標，而且兩端用的是完全不同的工作：

```
已確定          門檻在這段之間（未知）          已知證不出
BB(1)~BB(5)   6 ──────────────────────────────────────── 254    255, 256, ......
      往上推：確定 BB(6)、BB(7)......  →          ←  往下壓：造更小的 Z 型機器
```

- **往上推 (下界)**：逐一確定 BB(6)、BB(7) 的確切值。這要靠窮舉加逐台證明 (N3)，目前卡在 BB(6) 中行為類似考拉茲猜想的頑抗機器 (見 N7)。
- **往下壓 (上界)**：設計卡片更少、但行為仍獨立於 ZFC 的機器。這是程式設計與邏輯的壓縮工作，近年由 748 一路壓到 254。

Aaronson 曾猜測門檻不超過 20，但目前兩端相距仍很遠。除了這個門檻，研究者也關心其他問題：多符號版本 BB(n, k) 的數值、怪獸機器背後的數學，以及用 BB 替數學猜想標示難度 (N8)。

N6 拉多的提問：1962 年為什麼設計這場比賽 (起源與時點背景)

名稱的由來：忙碌海狸這個名字出自匈牙利裔美國數學家拉多 (Tibor Radó) 1962 年發表於《貝爾系統技術期刊》的論文〈論不可計算的函數〉(On Non-Computable Functions)，他把這個問題稱為「忙碌海狸遊戲」。拉多在論文中只說這是他發明的一個「邏輯遊戲」，用來教初學者認識圖靈機，並沒有解釋為何取這個名字，通行的解讀來自英語慣用語：busy beaver 與 busy as a beaver 都用來形容拚命工作、一刻不停的人，正好對應比賽要找的「在會停下的前提下忙得最久」的機器。常見的延伸聯想是海狸一根根搬木頭築壩，對應讀寫頭來回奔走、在紙帶上堆出一排 1，這是後人的形象化說法，並非拉多的說明。中文「忙碌海狸」是直譯，所以字面上看不出這層慣用語的意思。

時代背景：1936 到 1962：圖靈在 1936 年提出圖靈機，並證明停機問題無法判定 (N3)。此後數學家已經知道「不可計算的函數」存在，但當時的標準例子多半用**對角線論證**構造：先把所有程式排成一列，再造一個專門和第 n 個程式在第 n 題唱反調的對象 (N3 證明停機問題時的「唱反調」程式就是這種手法)。這類例子邏輯上嚴謹，卻相當抽象，要先接受「把所有程式排成一列」這一步。

拉多想要什麼：拉多在 1962 年的論文中，想給出更簡單、更具體的不可計算函數。他刻意避開「列舉所有可計算函數」的對角線手法，只依賴一個人人都能接受的原理：一個非空、有限的非負整數集合，一定有最大的元素。這正是 N2 證明冠軍存在時用的那一步。他把問題包裝成一場遊戲：在 n 張卡片的機器中，找出停機時留下最多個 1 的那台，成績記為 Σ(n)。

他證明了什麼：拉多證明 Σ(n) 長得比任何可計算函數都快，也就是對每一個可計算函數 f，n 夠大時 Σ(n) > f(n)，因此 Σ 不可計算 (N4 的機制二就是這條路線)。他同時證明了步數版本 S(n) 也不可計算。

拉多證明思路的白話版 (圖 8)：

1. **寫出 n 個 1 只需要 n 張卡片**：第 1 張卡寫 1、右移、換第 2 張，……，第 n 張寫 1 後停機 (同圖 4 的串接)。
2. **任何可計算函數都是一座固定大小的「工廠」**：f 可計算，代表有一台固定 c 張卡片的機器，能把紙帶上的 n 個 1 加工成 f(n) 個 1。工廠的卡片數 c 只跟 f 有關，跟 n 無關。
3. **把兩段接起來就是一台參賽機器**：前 n 張卡片寫出 n 個 1，再交給 c 張卡片的工廠，最後留下 f(n) 個 1 並停機。它共有 n + c 張卡片、會停，是合法參賽者，所以冠軍至少一樣好：Σ(n + c) ≥ f(n)。這個不等式的兩邊要分開讀：括號裡的 n + c 是「比賽組別」(這台機器用了幾張卡片，就參加幾張卡片的比賽)，右邊的 f(n) 是「這台機器的成績」(留下幾個 1)。Σ(n + c) 是 n + c 張卡片組所有會停機器中的最高分，最高分當然不低於其中任何一位參賽者的分數，所以至少是 f(n)；用「≥」而不用「=」，是因為冠軍可能遠比這台組裝機器厲害，我們只確定它不會更差。數字例子：若 f(n) = 2n、工廠需要 5 張卡片，取 n = 3 時，組裝機器有 3 + 5 = 8 張卡片、留下 6 個 1，所以 Σ(8) ≥ 6 (實際的 Σ(8) 大得多)。
4. **消掉多出來的 c**：c 是固定的額外成本，n 越大，它占的比例越小。拉多的技巧是讓工廠改做一件更狠的事 (例如算 f(2n))，工廠多出的這點卡片換來的好處遠超過成本，於是 n 夠大以後就得到 Σ(n) > f(n)。這一步的細節屬於技術性處理，白話版只需記得「固定成本會被 n 吃掉」。直覺上類似取極限時常數項變得不重要，但嚴格來說 c 並不是直接捨去，而是靠「讓工廠多做一點」的技巧換掉。

5. **推出不可計算**：如果 Σ 本身可計算，就可以拿 f(n) = Σ(n) + 1 當對手，它也可計算。依前四步，n 夠大時 Σ(n) > Σ(n) + 1，這不可能。所以 Σ 不可計算。這一步的關鍵是：「跟誰比都贏」也包括跟自己加一比，而沒有東西能贏過自己加一。論文發表時，只知道 Σ(1) = 1、Σ(2) = 4 已被證明，n = 3 的冠軍是否就是 6 個 1 仍未確定。

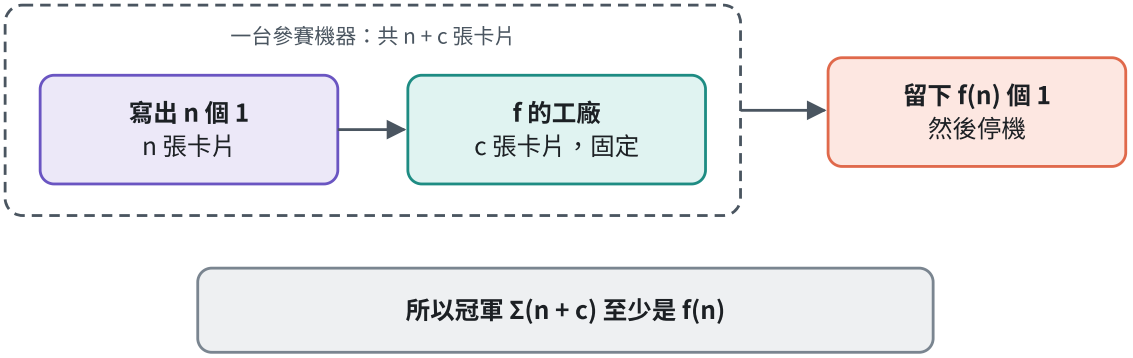


圖 8：拉多的組裝機器

看圖重點：n 張寫數字、c 張固定工廠，接起來就是合法參賽者。

從定義到追獵：拉多的原意是用遊戲說明「不可計算」，並沒有打算真的去算出每個值。但他把問題定義得如此具體，反而讓人忍不住想知道 BB(3)、BB(4) 到底是多少，從此開啟了延續六十多年的數值追獵 (N7)。

N7 追獵已知數值：BB(1) 到 BB(5)，與 BB(6) 的怪獸 (發展與應用)

每個 BB(n) 都分兩步完成 (圖 9)：第一步是**找到**一台跑很久的機器，這只給出下界 (冠軍至少這麼強)；第二步是**證明**沒有任何同組機器能更強，也就是把其餘機器全部判定為「停得更早」或「永遠不停」(N3 的窮舉加逐台證明)。第一步靠搜尋與運氣，第二步才是真正的難關，而且兩步之間的時間差越拉越長。

n	步數 S(n)	1 的個數 Σ(n)	找到冠軍	證明完成
1	1	1	1962 以前	1962 (拉多論文)
2	6	4	1962 以前	1962 (拉多論文)
3	21	6	1962	1965 (Lin 與拉多)
4	107	13	1966 (Brady)	1983 (Brady，1975 年先發表摘要)
5	47,176,870	4098	1989 (Marxen 與 Buntrock)	2024 (bbchallenge 協作，Rocq 驗證)
6	> 10↑↑10↑↑10↑↑8		2025-06 (mxdys，目前紀錄)	未完成

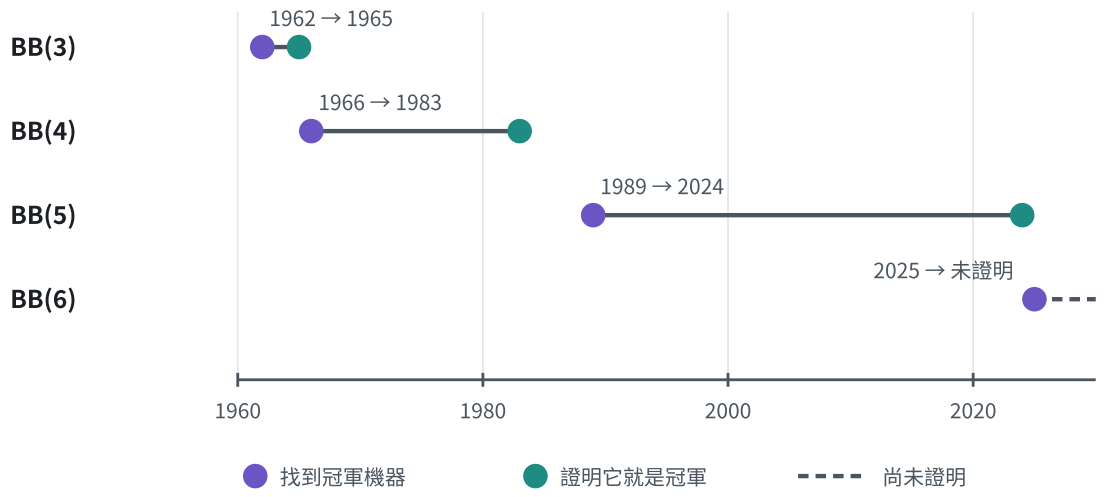


圖 9：冠軍找到與證明完成的時間差

看圖重點：證明所需時間從 3 年、17 年拉長到 35 年，BB(6) 尚未完成。

BB(5)：三十五年的證明：1989 年 Marxen 與 Buntrock 找到跑 47,176,870 步的機器後，大家普遍相信它就是冠軍，卻花了 35 年才證明。後來 bbchallenge 協作社群以判定器大批淘汰不停機的機器，剩下的少數難纏機器逐台寫出個別證明，全部論證放進 Rocq 驗證，於 2024 年宣布完成，正式論文 2025 年發表。

精確步數怎麼來的：只要手上有冠軍機器的規則表，步數就是「照規則一步步跑、每走一步計數加一、停機時讀出計數」，和 N1 手算 2 狀態冠軍跑 6 步完全相同，只是交給電腦做。圖靈機每一步都是確定的，沒有隨機，所以任何人重跑都會得到同一個數字。47,176,870 步對電腦來說很少：在本單元製作時，用一個約二十行的 C 程式模擬這台機器，約 0.2 秒跑完，結果正是 47,176,870 步、停機時紙帶上 4098 個 1。BB(5) 冠軍的規則表如下 (原始記法 1RB1LC_1RC1RB_1RD0LE_1LA1LD_1RZ0LA)：

卡片	腳下是 0	腳下是 1
A	寫 1，右移，換 B	寫 1，左移，換 C
B	寫 1，右移，換 C	寫 1，右移，換 B
C	寫 1，右移，換 D	寫 0，左移，換 E
D	寫 1，左移，換 A	寫 1，左移，換 D
E	寫 1，右移，停機	寫 0，左移，換 A

所以「算出步數」從來不是難處。難的是兩件事：一是從六十多兆台機器中**找到**這台 (Marxen 與 Buntrock 靠電腦搜尋大量候選機器，並用加速模擬的技巧跳過重複的長段落)；二是**證明**其他機器沒有一台能停得更晚，這一步花了 35 年。

BB(6) 的怪獸機器：6 狀態的機器中，有一些頑抗機器的行為等價於一道未解的數學題，被社群稱為**怪獸機器** (Cryptid)。最有名的一台叫 **Antihydra** (2024 年 6 月由 mxdys 回報)。它的行為可以翻譯成一個數字遊戲：從 4 開始，反覆把數字 n 換成「n 的 1.5 倍取整數再加 2」，同時統計出現過幾次奇數、幾次偶

數，如果某個時刻奇數出現的次數超過偶數次數的兩倍，機器就停下。這種「乘 3 除 2、看奇偶」的規則，跟著名的**考拉茲猜想** (Collatz conjecture，任取正整數，偶數就除以 2、奇數就乘 3 加 1，猜想最後一定會到 1，至今無人能證明) 屬於同一類難題。電腦模擬顯示 Antihydra 的計分像一場「每步 +2 或 −1、機率各半」的隨機漫步，平均一路往上，跌破零的機率小到難以想像，所以大家相信它永遠不停；但「看起來幾乎不可能」不等於證明。只要 Antihydra 這類機器還證不出來，BB(6) 就無法確定。

怪獸機器的定義：bbchallenge wiki 的定義是：從全 0 紙帶出發時，行為可以被一條簡單規則**完整描述**，而那條規則屬於一類未解、而且公認困難的數學問題 (例如考拉茲類問題) 的機器。wiki 也註明這個定義帶有一些主觀成分。關鍵在「完整描述」：怪獸機器並不是「看不出它在幹嘛」，而是已經完全看懂它在做什麼 (例如 Antihydra 就是在玩那個 1.5 倍加 2 的數字遊戲)，卻因為那個數字遊戲本身是數學難題而無法判定。

類別	我們對它的了解	卡在哪裡
已判定的機器	已證明會停 (並知道步數) 或永遠不停	無
一般頑抗機器	還沒找到合適的分析方法	缺技術，可能只是還沒人仔細研究
混亂但非怪獸的機器	行為看似雜亂，也還說不清規則	缺技術，且沒對應到已知的數學難題
怪獸機器	規則已完全看懂	規則本身是未解的數學難題

怪獸機器也出現在其他組別：多符號的 BB(3, 3) 有 Bigfoot (2023 年 11 月)，BB(2, 5) 有 Hydra (2024 年 5 月)。5 狀態 2 符號的組別目前沒有列出怪獸機器，這也是 BB(5) 能夠完成證明的原因之一。截至 2026 年 10 月，6 狀態機器在扣除等價者後仍有約 740 台頑抗機器待處理。

AI 與證明輔助工具的角色：這個領域常同時出現兩種工具，要分清楚。第一種是**證明輔助工具** (例如 Rocq，舊名 Coq；以及 Lean)，它本身不是 AI，而是一個嚴格檢查證明的程式：人把證明一步步寫成它看得懂的形式 (這個過程稱為**形式化**)，它逐步核對，每一步都正確才會接受。2024 年 bbchallenge 協作社群證明 BB(5) = 47,176,870，就是把全部論證寫進 Rocq 由它驗證，靠的是協作者與傳統程式，並非 AI。第二種是語言模型這類 AI，近一兩年開始被用來協助：

- 2026 年 2 月，bbchallenge 發起人 Tristan Stérin 測試 Claude Opus 4.6 撰寫 Rocq 證明。他把既有 BB(4) 證明的 260 條引理打亂名稱後請模型補上證明，模型約 10 小時證出 258 條，另有 2 條未完成。他也提醒：原證明可能在模型的訓練資料中，而且證明的整體架構是人先搭好的，模型負責填入細節。
- 2026 年 10 月 13 日，bbchallenge wiki 收錄一台 254 狀態、行為獨立於 ZF 集合論的機器 (意義見 N5)，紀錄寫明由 Mira 在 ChatGPT 協助下找到，再由 Jeff Ketchersid 在 Claude 協助下用 Rocq 形式化。這是當時最小的紀錄 (前一紀錄為 2025 年 8 月的 432 狀態)，wiki 未說明兩人與 AI 各自分工的細節，也未記載驗證檔之外的獨立審查。

AI 的角色目前是協助搜尋候選機器、草擬證明，最終的正確性仍由證明輔助工具逐步核對來保證。

窮舉與 AI 的分工：逐台檢查機器確實適合交給電腦，但這部分主要由傳統程式負責。BB(5) 的證明中，社群寫了一批稱為**判定器** (decider) 的程式，每一種專門辨認一類「明顯不會停」的模式 (例如機器陷入完全重複的循環、或一邊重複一邊往同一方向平移)，一次就能處理大量機器，而且每個判定器本身的正確性也經過 Rocq 驗證。傳統程式的優點是結果可以完全核對；語言模型會出錯，不適合直接負責「保證沒有遺漏」的工作。判定器處理不掉的少數頑抗機器，需要新的數學想法，這才是 AI 可能派上用場的地方。不過 AI 本身也是程式，N3 的結論對它同樣成立：不存在能對所有 n 證完全部不停機機器的通用方法，AI 只能協助個別的機器，無法一勞永逸。

N8 用忙碌海狸替數學猜想標難度 (發展與應用)

直覺：N5 用一台「找矛盾」的機器 Z，把「ZFC 會不會自相矛盾」翻譯成「Z 會不會停」。同一招可以套到許多數學猜想上：只要能寫一台「找反例，找到才停」的機器，猜想就變成停機問題，而知道對應的 BB 值就能回答它。於是每個猜想都可以標上一個刻度：最少需要幾張卡片才能把它寫成機器 (圖 10)。

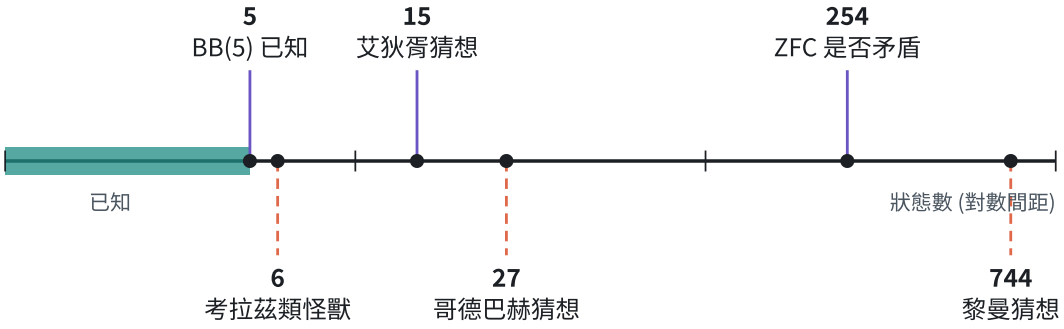
機制：以哥德巴赫猜想為例：**哥德巴赫猜想** (Goldbach conjecture，1742 年提出) 說，每個大於 2 的偶數都能寫成兩個質數的和，至今未被證明。2026 年 9 月有報導指出，一個名為 GPT-6 Astra 的 AI 系統證明了它的弱化版本「劉維爾版哥德巴赫猜想」(把「兩個質數」放寬成「兩個質因數個數為奇數的整數」，質因數重複也要算；例如 10 = 2 + 8 在這個版本中成立，因為 8 = 2 × 2 × 2 有 3 個質因數，但 8 不是質數)，並以 Lean 4 形式化；報導來源多為二手轉述，原始宣布者為匿名帳號。各報導都明確表示，經典的哥德巴赫猜想仍未解決，所以上面 27 狀態機器的意義不受影響。

1. 造一台機器 G：依序檢查 4、6、8、10……，對每個偶數嘗試所有拆成兩數相加的方式，看有沒有兩個都是質數的；只要碰到一個偶數怎麼拆都不行，就停機。所以 G 會停 ⇔ 哥德巴赫猜想是錯的。
2. 假設 G 有 k 張卡片，而我們知道 BB(k)：讓 G 跑 BB(k) 步。已經停了，猜想錯；還沒停，就永遠不會停 (N5 第 3 步的道理)，猜想對。
3. 所以「知道 BB(k)」等於「解決了這個猜想」，k 越小，代表這個猜想越容易被「裝進」一台小機器。

刻度表 (狀態數為目前已知、能編碼該問題的最小機器之一)

問題	內容	狀態數	出處
艾狄胥猜想	2 的 n 次方 (n > 8) 寫成三進位時，至少有一位數字是 2	15	Stérin 與 Woods，2021
哥德巴赫猜想	每個大於 2 的偶數是兩個質數之和	27	「Code Golf Addict」，2015 (2021 年時尚未正式發表)
ZFC 是否自相矛盾	見 N5	254	2026-10 紀錄

問題	內容	狀態數	出處
黎曼猜想	關於質數分布的著名未解猜想	744	Aaronson、Matiyasevich、O’Rear，2016 (2021 年時尚無發表的完整證明)



刻度為機器的狀態數；知道該 BB 值，就能判定對應的問題

圖 10：忙碌海狸尺

看圖重點：已知的範圍止於 5，而 15、27 就已經是著名猜想。

邊界：這把尺只是「標難度」，並沒有讓猜想變簡單。BB(5) 已經花了 35 年，BB(6) 又卡在考拉茲類的怪獸機器，要算出 BB(15) 或 BB(27) 在實務上遙不可及。它的意義在於提供一個共同單位：原本性質完全不同的猜想，被放在同一條尺上比較，而且讓人看清楚，6 張卡片就已經碰到數學未解之謎的邊緣。這招也只適用於「找到反例就能確認」的猜想，也就是反例可以逐一檢查的類型。

這把尺量的是什麼：它像是替猜想統一了一套度量衡，但量的是「這個問題最少要用幾張卡片才能完整寫下來」，比較接近問題的**描述長度**，而不是人類證明它的難度。三點限制要記得：一是刻度只是目前紀錄、屬於上界，有人寫出更精簡的機器，數字就會往下掉 (ZFC 的數字十年間從 7910 掉到 254)；二是刻度小不代表對人類容易，艾狄胥猜想只要 15 張卡片，卻同樣無人能證；三是只能量「反例可逐一檢查」的猜想。比喻：它像用「裝得下這件物品的最小箱子」來比較物品，箱子大小有統一單位，卻不等於物品的重量或價值。

常見誤解

誤解	正確理解	為什麼容易誤解
函數每個值都有確定答案，就是「可計算」 (學前診斷 Q2)	可計算要求同一個程式能對所有 n 在有限時間算出答案；BB 每個值都確定存在，卻沒有這樣的程式 (N2、N3)	日常語感中「有答案」就等於「算得出來」；對單一 n 永遠存在「直接印出答案」的程式，讓人忽略「同一個程式通吃所有 n」的要求

術語表

術語	原文	別名	白話定義	首次出現
圖靈機	Turing machine		一條無限長的 0／1 紙帶加上一個讀寫頭，依照一張規則表一步步讀、寫、移動的極簡計算機模型	N1
狀態	state	卡片	規則表中的一列，規定讀寫頭看到 0 或 1 時要寫什麼、往哪走、下一步換到哪一列	N1
停機	halt		機器走到特殊狀態 H，從此不再動作；H 是去處，本身沒有規則列	N1
可計算函數	computable function	可計算	存在同一個程式，輸入任何 n 都能在有限時間內輸出 f(n) 的函數	N3
有限時間	finite time	有限步數	程式在某個有限的步數後停下；不要求事先知道步數，也沒有統一上限，只排除「永遠不停」	N3
封閉公式	closed-form expression	公式解	由有限次加減乘除、次方等明確運算組成、代入數字就能算出答案的公式	N3
頑抗機器	holdout		窮舉時跑了很久仍未停機、必須另行證明它永遠不停的機器	N3
阿克曼函數	Ackermann function	A(m, n)	用層層套疊的遞迴定義、仍可計算，但成長比任何固定高度的指數塔都快的函數	N4
迭代冪次	tetration	↑↑、高德納箭號	把次方重複疊起來：10↑↑3 = 10 的 (10 的 10 次方) 次方	N4

術語	原文	別名	白話定義	首次出現
對角線論證	diagonal argument	對角線法	把所有對象排成一列，再造一個與第 n 個在第 n 處刻意不同的新對象，用來證明某種東西不存在或不在列表中	N6
公理系統	axiom system	公理	一組不加證明、大家同意作為起點的基本規則，所有證明都從它出發	N5
ZFC	Zermelo-Fraenkel set theory with Choice	ZF 加選擇公理	現代數學最常用的公理系統，幾乎所有定理都能在其中證明	N5
獨立於	independent of	不可判定	某命題在某公理系統中既無法證明為真、也無法證明為假	N5
哥德爾第二不完備定理	Gödel’s second incompleteness theorem		夠強且不自相矛盾的公理系統，無法在內部證明自己不自相矛盾 (1931)	N5
怪獸機器	Cryptid		從全 0 紙帶出發的行為可被簡單規則完整描述，而該規則屬於未解且公認困難的數學問題的機器；已看懂、卻因數學難題而無法判定	N7
考拉茲猜想	Collatz conjecture	3n + 1 猜想	任取正整數，偶數除以 2、奇數乘 3 加 1，反覆操作最後一定會到 1；至今未被證明	N7
哥德巴赫猜想	Goldbach conjecture		每個大於 2 的偶數都能寫成兩個質數之和；1742 年提出，至今未證	N8
黎曼猜想	Riemann hypothesis		關於質數分布的著名未解猜想 (1859 年提出)	N8

術語	原文	別名	白話定義	首次出現
艾狄胥猜想 (三進位)	Erdős conjecture on powers of 2		n > 8 時，2 的 n 次方寫成三進位至少有一位數字 2	N8
判定器	decider		專門辨認某一類「明顯不會停」模式的程式，用來大批淘汰不停機的機器	N7
證明輔助工具	proof assistant	Rocq (舊名 Coq)、Lean	逐步嚴格檢查數學證明的程式，本身不是 AI；每一步都正確才接受	N7
形式化	formalization		把證明一步步改寫成證明輔助工具能逐條核對的形式	N7
停機問題	halting problem		判斷任意程式最終會不會停下的問題；圖靈 1936 年證明沒有程式能對所有情況正確判斷	N3
迴圈	loop	繞圈子	機器依同一組卡片反覆執行同一段動作；迴圈裡可以再套迴圈	N2
忙碌海狸函數	Busy Beaver function	BB(n)、拉多函數；兩種版本 Σ(n) 與 S(n)	所有會停下的 n 狀態圖靈機中，最大的成績 (步數或 1 的個數)	N1

附錄 A：學習紀錄

學前診斷

三題基本概念中，停機問題原本不熟悉，「可計算」誤選為「每個 f(n) 都有確定值」，成長速度答對；選擇深入學習，維持 8 個節點。

提問紀錄

#	使用者提問 (摘要)	回答重點	歸位
1	學前診斷作答	Q1 不確定、Q2 選「有確定值」、Q3 正確；選深入	僅紀錄

#	使用者提問 (摘要)	回答重點	歸位
2	為什麼叫海狸	拉多 1962 年命名，本人未留下解釋；通行解讀為英語慣用語 busy beaver「拚命工作的人」	整併至 N6
3	繼續教學：N2	有限機器、剔除不停機、取最大值三步論證	整併至 N2
4	多狀態時增加的是什麼；確認公式是上限	公式為固定 n 的精確總數；H 是去處不是卡片；增加的是格數與「換卡」選項，意義在於內部記憶變多、可組出套疊的迴圈	整併至 N2
5	沒聽懂「多一張卡片的意義」，請舉例	里程表比喻 (G3)；內圈、外圈的卡片分工；冠軍步數 1、6、21、107、47,176,870	整併至 N2
6	里程表沒聽懂；推測「多狀態 = 四種動作的卡片各變成 2、3、4 張」	四種動作組合正確；卡片是編號步驟，差別在「下一步去哪」；1 張與 3 張卡片串接例 (G4)；箭頭指回前面形成迴圈	整併至 N2
7	推測「卡片像撲克牌花色，照排序好的固定順序抽」	對：過程完全確定、無隨機。修正：每張卡有兩個分支，下一張由腳下符號決定；同一張卡可反覆回來；改用冒險遊戲書比喻；以 G1 中卡片 B 三次出場說明	整併至 N2
8	繼續教學：N3	補「可計算」與「有確定值」、停機問題；若能算 BB 則能判定停機，故 BB 不可計算	整併至 N3
9	「有限時間」怎麼定義 (教材摘錄提問)	以步數計；對每個 n 在某個有限步數停下即可；無統一上限、不需事先知道、再久都算；只排除永遠不停；「合理時間」屬計算複雜度	整併至 N3

#	使用者提問 (摘要)	回答重點	歸位
10	BB(n) 有沒有標準數學式；是否必須窮舉	有嚴格定義 (取最大值的集合式)，沒有封閉公式 (否則可計算)；實務是窮舉＋逐台證明頑抗機器不停，證明無通用方法	整併至 N3
11	有人用 AI 協助過嗎	區分證明輔助工具與 AI；BB(5) 證明靠 Rocq 非 AI；2026-02 Opus 4.6 補 BB(4) 證明 258/260；2026-10 254 狀態 ZF 獨立機器 (ChatGPT 協助尋找、Claude 協助形式化)	整併至 N7
12	推測：逐台窮舉、找證據證明不停，應是 AI 擅長的事	方向部分對：逐台檢查交給電腦，但主力是可完全核對的傳統判定器；AI 適合協助少數頑抗機器；AI 也是程式，受 N3 限制	整併至 N7
13	繼續教學：N4	可計算函數無法永遠壓住 BB；把 f 的程式裝進參賽機器得 BB 最終勝出；小 n 時阿克曼較大	整併至 N4
14	f(n) 是否隨 n 不同；以 BB(2) = 6 為例說明「它的 f(n)」	f 是 BB 的對手 (任一可計算函數)，是一條固定規則，值隨 n 變而規則不變；BB 沒有自己的 f；n = 2 時 n^2、2^n 得 4 輸給 6，阿克曼 7 贏	整併至 N4
15	推測：目標是找 f(n) > BB(n)，再用 f(n) 步數反推 BB(n)	流程邏輯正確，但這樣的 f 不存在 (否則 BB 可計算)；N4 是成長速度定理而非求值方法；單一 n 的上限只能靠逐台證明取得	整併至 N4
16	機制一的假設與「遲早被 BB 超過」是否同一件事	不同：機制一推得 BB 無限多次超過 f (可時贏時輸)；機制二推得某處之後每一項都超過 (更強)；以甲乙兩種情形對照	整併至 N4

#	使用者提問 (摘要)	回答重點	歸位
17	沒聽懂機制一與機制二的差別	下雨天比喻：「總會再遇到雨天」對「某天起天天下雨」；圓點圖 G6	整併至 N4
18	確認：BB(n) 大方向是永遠比某個可計算函數大	方向對；修正兩處：「某個」改「每一個」，「永遠」改「n 夠大之後」；起點 N 隨 f 而變	整併至 N4
19	質疑：結論看似無用，大小不是看跟誰比嗎	單一 n 確實看對手；定理價值在與對手無關，BB 位於「誰比誰快」整條鏈之上；後果：無法用公式估 BB、程式極限無法被程式預測、為 N5 與 N8 鋪路	整併至 N4
20	繼續教學：N5	機器 Z 找 ZFC 矛盾；若 ZFC 證出 BB(k) 則證明自身一致，違反哥德爾；三種「做不到」對照；k 的紀錄 7910 → 254	整併至 N5
21	為什麼跑 V 步沒停就能斷定 Z 永遠不停	Z 也是 k 狀態參賽者；BB(k) = V 表示所有會停者都在 V 步內停，晚停即打破紀錄；k = 2 實例；路跑最慢完賽者比喻	整併至 N5
22	確認：目前的目標是找出最小的不可證 k 嗎	是方向之一；兩端夾擠：往上確定 BB(6) 等確切值、往下造更小的獨立機器；另有多符號版、怪獸機器、猜想標難度等方向	整併至 N5
23	繼續教學：N6	1936 圖靈到 1962 拉多；當時例子多用對角線論證；拉多只靠「有限集合有最大值」造出具體不可計算函數，證明 Σ 勝過所有可計算函數；遊戲用於教初學者	整併至 N6

#	使用者提問 (摘要)	回答重點	歸位
24	請白話補充拉多的證明思路	n 張卡寫 n 個 1；f 是 c 張卡的固定工廠；接成 n + c 張的參賽機器得 Σ(n + c) ≥ f(n)；固定成本被 n 吃掉；以 f = Σ + 1 推出矛盾	整併至 N6
25	卡在第 3 步「Σ(n + c) ≥ f(n)」；自述第 4 步像取極限、第 5 步是 n > n + 1	括號內是組別、右邊是成績；最高分不低於任一參賽者；≥ 因冠軍可能更強；n = 3、c = 5 數字例；第 4 步直覺正確但 c 靠技巧換掉而非直接捨去	整併至 N6
26	繼續教學：N7	找到與證明兩步；BB(1) 到 BB(6) 時間表；BB(5) 35 年證明；Antihydra 與考拉茲類難題；約 740 台頑抗機器	整併至 N7
27	47,176,870 這麼精確的步數怎麼算出來	照規則逐步模擬並計數，確定性所以可重現；本單元以 C 程式 0.2 秒重跑得 47,176,870 步、4098 個 1；附冠軍規則表；難處在找到與證明	整併至 N7
28	怪獸機器的定義；是不是找不出循環或結束點	定義：行為可被簡單規則完整描述、而規則屬未解難題；已完全看懂才算怪獸，與「還沒看懂」的頑抗機器不同；四類對照表；其他組別的 Bigfoot、Hydra；BB(5) 無怪獸	整併至 N7
29	繼續教學：N8	找反例機器把猜想變成停機問題；哥德巴赫 27、艾狄胥 15、ZFC 254、黎曼 744；只標難度不降難度；限反例可逐一檢查的猜想	整併至 N8
30	題外話：哥德巴赫猜想好像被 GPT 證明了	2026-09 報導證明的是弱化的劉維爾版 (Lean 4 形式化，來源為二手轉述)；經典版仍未解	整併至 N8

#	使用者提問 (摘要)	回答重點	歸位
31	像是統一了難度的度量衡嗎	是共同單位，但量的是描述長度而非證明難度；刻度是可下降的上界；小刻度不等於容易；限反例可檢查的猜想；最小箱子比喻	整併至 N8

附錄 B：資料來源

- Tibor Radó, *On Non-Computable Functions*, Bell System Technical Journal 41(3), 1962, https://www.gwern.net/doc/cs/computable/1962-rado.pdf：N6 拉多的動機 (避開對角線、只用有限集合有最大值)、遊戲用於教學、Σ 勝過所有可計算函數、當時已知 Σ(1)、Σ(2)
- Wikipedia, *Busy beaver*, https://en.wikipedia.org/wiki/Busy_beaver：N6 拉多 1962 年論文名稱與期刊
- bbchallenge wiki, https://wiki.bbchallenge.org：N2 冠軍步數 S(5) = 47,176,870
- Stanford Encyclopedia of Philosophy, *Turing Machines*, https://plato.stanford.edu/entries/turing-machine/：N3 停機問題 (圖靈 1936)
- Tristan Stérin, *Claude Opus 4.6 formal proofs* (2026-02-17), https://tristan.st/blog/opus_4_6_formal_proofs：N7 BB(4) 證明實驗
- bbchallenge wiki, *Logical independence*, https://wiki.bbchallenge.org/wiki/Logical_independence：N7 254 狀態機器紀錄 (2026-10-13)
- bbchallenge wiki, Main Page, https://wiki.bbchallenge.org/wiki/Main_Page：N7 BB(5) 於 2024 年以 Rocq 證明
- bbchallenge wiki, *BB(6)*, https://wiki.bbchallenge.org/wiki/BB(6)：N4 BB(6) 下界 > $10\uparrow\uparrow 10\uparrow\uparrow 10\uparrow\uparrow 8$ (mxdys, 2025-06)
- bbchallenge Collaboration, *Determination of the fifth Busy Beaver value* (2025), https://arxiv.org/abs/2509.12337：N7 判定器方法
- bbchallenge wiki, *Logical independence*：N5 Aaronson 猜測門檻 ≤ 20
- Marxen 等人歷史整理, https://mrob.com/pub/math/bb-history.html：N7 BB(3)、BB(4)、BB(5) 冠軍年份
- Pascal Michel, https://bbchallenge.org/~pascal.michel/adv：N7 Brady 1983 BB(4) 證明、Marxen 與 Buntrock 1990
- bbchallenge wiki, *Antihydra*, https://wiki.bbchallenge.org/wiki/Antihydra：N7 Antihydra 規則與隨機漫步論證
- bbchallenge wiki, *Cryptids*, https://wiki.bbchallenge.org/wiki/Cryptids：N7 怪獸機器定義、Bigfoot、Hydra

- Quanta Magazine, *How the Slowest Computer Programs Illuminate Math’s Fundamental Limits* (2020-12-10), https://www.quantamagazine.org/how-the-slowest-computer-programs-illuminate-maths-fundamental-limits-20201210/：N8 哥德巴赫 27 狀態 (2015)、黎曼 744 狀態 (2016)
- Stérin & Woods, *Hardness of busy beaver value BB(15)* (2021), https://arxiv.org/abs/2107.12475：N8 艾狄胥猜想 15 狀態，及 27、744 尚未正式發表的說明
- 36Kr (2026-09-21), https://eu.36kr.com/en/p/3992525310802946；KuCoin News (2026-09-21), https://www.kucoin.com/news/flash/gpt-6-astra-proves-linnik-s-variant-of-goldbach-conjecture：N8 劉維爾版哥德巴赫猜想報導，經典版仍未解
- Scott Aaronson, *The Busy Beaver Frontier* (2020), https://www.scottaaronson.com/papers/bb.pdf：N1 的兩種計分方式與 2 狀態冠軍